Optional Walled Gardens Through Progressive Web Apps

Anonymous Author(s)

ABSTRACT

Traditionally, desktop operating systems such as Windows, macOS, and Linux have allowed sideloading of applications and lacked app sandboxing, resulting in numerous security vulnerabilities, especially for non-tech-savvy users. In contrast, mobile platforms like Android and iOS employ app sandboxing and typically require apps to be installed through official app stores. However, these mobile platforms are often criticized for being overly restrictive, creating *walled garden* ecosystems. Despite these restrictions, many security threats, such as phishing websites, still persist. Furthermore, power users find these environments frustrating due to their inability to customize or operate outside the constraints imposed by the ecosystem.

With the emergence of Progressive Web Apps, this paper explores a novel security model called the S/P Login Toggle. This approach aims to segregate the use cases for power users and average users by leveraging the flexibility and security of PWAs. The paper provides a historical overview, compares the S/P Login Toggle with the security models of existing operating systems, and demonstrates its superiority in balancing usability and security for a diverse range of users.

1 INTRODUCTION

For most of computing history, users enjoyed broad freedom to install, modify, and repair software and hardware on their own devices. This freedom is closely tied to the *right to repair* and the traditional understanding that personal property belongs to its owner. With the rise of the Internet in the 1980s and 1990s, however, this freedom increasingly collided with security risks. Non-technical users could easily download malware[28], fall for phishing[27], or compromise entire systems. Nearly all Windows users now rely on antivirus software, and the 2024 CrowdStrike incident demonstrated the fragility of this model: a single signed update caused global blue screens and disrupted critical infrastructure[9].

The introduction of the iPhone in 2007 accelerated a shift toward *walled gardens*. Users could no longer run arbitrary binaries, install alternative operating systems, or bypass app stores[2, 26]. Apple and Google now control app distribution and take a 30% cut[14, 19]. Stallman and the FSF describe this as `tivoization`[36]. Governments have responded with antitrust actions[7, 12, 31], yet both app stores still host large numbers of scam and predatory applications[24, 54].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, July 2017, Washington, DC, USA
© 2026 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

These restrictions extend far beyond app stores. Companies increasingly deploy software locks, cryptographic pairing, and proprietary components to undermine repair rights. The DMCA is routinely used to block legitimate repair and research[15]. Groups such as Keep Android Open[23] document how Android devices are becoming more locked down despite being marketed as "open." Regulators such as the FTC and FDA have found "scant evidence" supporting Big Tech's claims that these restrictions are necessary for safety; they primarily preserve ecosystem control[48].

Recent events highlight this fragility. Google's 2025 developer verification rollout broke legitimate apps and locked out independent developers, illustrating how a single platform decision can destabilize the entire Android ecosystem. This is part of a broader pattern of *enshittification*, where platforms degrade over time to maximize control and revenue.

These practices also worsen the global e-waste crisis. A 2023 PIRG report found that U.S. households spend $1,767 annually on electronics—a 19% increase since 2019—and could save $49.6 billion per year with better repair access. The WHO reports that only 22.3% of global e-waste is formally recycled. A CBS News investigation[6] documented how more than 50 million tons of e-waste are produced annually, most of which is exported to the global South, where informal recyclers sift through toxic debris without protection. Researchers have found heavy-metal contamination even in the breast milk of mothers in affected African communities. In China, independent estimates suggest the real recycling rate is below 8%, with entire villages suffering cancer clusters due to informal e-waste processing. Locked-down, non-repairable devices directly fuel this environmental harm.

A parallel trend accelerating the erosion of digital autonomy is the industry's push toward cloud dependence. As the saying goes, "the cloud is just someone else's computer,"[22] yet major vendors increasingly design products that require remote servers for basic functionality. The fragility of this model became clear on March 1, 2026, when an Amazon Web Services data center in the United Arab Emirates was struck by unidentified objects during a period of intensified regional conflict involving Iran, the United States, and Israel. The impact triggered sparks and a fire, leading emergency crews to cut power and causing outages across multiple availability zones. AWS did not confirm whether the incident was directly linked to Iranian missile and drone strikes, but the outage occurred amid broader retaliatory attacks across the Gulf region. This event underscored how cloud dependence centralizes risk: a single facility failure can disrupt critical services across entire regions.

This centralization reflects a broader shift toward subscription models, remote execution, and platform-controlled ecosystems. The World Economic Forum's 2016 scenario essay, which popularized the phrase "You will own nothing and you will be happy,"[55] has become a widely cited symbol of public concern that personal ownership is being replaced by perpetual rental. The trend is visible across modern computing: devices require cloud authentication,

applications depend on vendor accounts, and essential functions run on remote servers rather than locally. Scholars describe this structure as a form of techno-feudalism, in which users rely on platform landlords who control access, updates, and even continued device operation. Combined with the physical centralization of cloud infrastructure—where a single data-center outage can disrupt entire regions—this shift erodes the autonomy and resilience that once defined personal computing.

Even the supposed "security" benefits of locked ecosystems are overstated. Signed binaries still fail: the CrowdStrike update was signed, kernel anti-cheat drivers are abused for surveillance, and projects like Notepad++ reject mandatory signing due to centralization risks[**?**]. Rising hardware prices further erode consumer ownership.

At this point, it becomes clear that **security is only the surface cause**. The **root cause is the erosion of personal ownership**. As Perzanowski and Schultz argue in *The End of Ownership*[**?**], intellectual property law has expanded beyond its intended scope and now routinely overrides traditional personal property rights. Buying two identical devices does not grant the right to repair one with parts from the other; companies claim that software licenses supersede ownership. This inversion of property rights is a central driver of enshittification across the tech industry.

These dynamics have long-term consequences. Windows cannot adopt the same restrictions as mobile platforms because developers do not target the Microsoft Store at comparable scale. Windows S mode[44] lacks apps, and Windows users strongly prefer open software distribution[47, 50]. Meanwhile, many Android devices stop receiving updates after only a few years[20], leaving outdated kernels and unpatched vulnerabilities[34].

The common narrative that users must trade freedom for security is itself misleading. Even the surface explanation—that locked-down ecosystems are necessary for safety—does not withstand scrutiny. As shown throughout this section, these restrictions routinely fail to deliver meaningful security benefits, while simultaneously blocking repair, modification, and user autonomy. The deeper issue is not security at all, but the steady dismantling of personal ownership in favor of corporate control. Users have been losing authority over their hardware, software, and repair rights[5], not because this loss makes devices safer, but because it strengthens the power of platform gatekeepers. This paper argues that the supposed tension between security and usability is a false dilemma[18]: we can design systems that provide strong protection without sacrificing ownership, openness, or user freedom.

With the rise of Progressive Web Apps (PWAs)[13, 53], many services—including Office, YouTube, TikTok, Spotify, Instagram, Discord, Reddit, LinkedIn, VSCode, Copilot, ChatGPT, and more—now offer PWA versions. PWAs run inside browser sandboxes, reducing risk compared to native Win32 binaries. This makes it possible to design an *optional walled-garden mode* for non-technical users while preserving full openness for power users.

The S/P mode proposed in this paper aims to show that the supposed trade-off between security and freedom is false. Users can have both: a safe, simplified environment for everyday use and a fully open, repair-friendly, developer-friendly environment for those who need it—without surrendering ownership or falling into enshittified ecosystems.

2 BACKGROUND

2.1 `unsafe` keyword in C#/Rust

In programming languages like C# and Rust, pointer operations are explicitly marked as `unsafe`. This establishes a language-wide boundary to distinguish memory-unsafe code. It is a brilliant concept because it allows users to demonstrate that they can handle APIs correctly, reducing the likelihood of errors. This is especially valuable when working with third-party libraries. In contrast, while C++ employs RAII (Resource Acquisition Is Initialization) for resource management, it does not inherently ensure memory safety. This is because C++ libraries cannot guarantee that users will utilize them correctly 100% of the time.

The S/P toggle concept discussed in this paper draws inspiration from the `unsafe` keyword in C# and Rust. It aims to create a similar boundary, enabling users to safely and correctly operate their devices. At the same time, it ensures that advanced users retain the flexibility they need to perform tasks when necessary.

2.2 Google's Misplaced Blame on Memory-Unsafe Languages

Google Chromium has consistently identified memory-unsafe languages, such as C++, as a primary source of its security issues[35]. However, this paper contends that, based on this same logic, Google Chrome itself should be deemed fundamentally unsafe. The root cause of these issues extends beyond programming languages, stemming instead from the inherent design of web browsers. As of February 2025, Google Chrome held a dominant 66.3% share of the global market[1], with Chromium-based browsers collectively exceeding 80%. This extensive exposure to arbitrary and untrusted input highlights the fundamental weaknesses in browser architecture.

Every website accessed via a browser is an untrusted remote code execution. While memory-unsafe languages do contribute to some security problems, the larger issue lies in the browser's unavoidable handling of untrusted input, which makes it significantly more susceptible to threats compared to other applications. Following this logic, banning Google Chrome for security concerns for majority of users would align with the rationale used for criticizing C++. If web browsers never existed in the world, the majority of security issues would also disappear, as most devices would no longer face the risks associated with exploiting security vulnerabilities.

2.3 App Sandboxing

App sandboxing confines an application's access to system resources and user data within a restricted environment. Apple mandates that all apps in the App Store utilize app sandboxing[3]. In the Linux ecosystem, app sandboxing has also gained significant traction with technologies like Flatpak, Snap, and AppImage. Similarly, all Android apps are sandboxed by default. The primary advantage of app sandboxing is its ability to enforce app-specific permissions, limiting what each application can do.

However, the drawbacks are substantial. Applications often need to be rewritten to function within sandboxed environments, and this requirement extends to their third-party library dependencies. Many open-source libraries and software lack the necessary updates to adapt to these environments. Additionally, performance can

suffer due to the sandbox's isolated nature, which restricts resource sharing. A notorious example is Firefox on Ubuntu, distributed as a Snap package. The Snap version of Firefox has been criticized for slow boot time—sometimes taking up to 20 seconds on an average computer—because it bundles its own version of glibc and other libraries instead of using libraries the distribution itself provides[8, 46].

These sandboxed apps can also create conflicts. Drivers bundled within Snap or Flatpak applications may not align with those provided by the operating system, leading to potential compatibility issues. Furthermore, while the app itself is sandboxed, it still operates within the same memory address space as the kernel. If the kernel has vulnerabilities, app sandboxing cannot prevent kernel-level exploits. This limitation is evident on Android, where malware remains a persistent issue despite widespread app sandboxing. The problem is particularly relevant for mainstream OS kernels like Windows NT, Linux, and Darwin, all of which employ monolithic kernels, not micro kernels.

2.4 Progressive Web Apps

Progressive Web Apps (PWAs) are essentially websites that function as applications. Since every platform includes browser support and mainstream web browsers embrace this technology, PWAs are highly versatile. For example, websites like Snaeplayer Music Player or Flow EPUB Reader can be accessed, added to your home screen on any device—be it a phone or PC—and effectively installed as apps. These apps operate using the browser engine already available on the device.

Unlike WebView apps or Electron apps, PWAs do not bundle their own Chromium installation. WebView apps, such as the pre-installed Microsoft Outlook on Windows 11, and Electron apps often duplicate Chromium, which can be resource-intensive. In contrast, PWAs share the same browser engine that the user is already utilizing.

Progressive Web Apps support offline functionality once installed. For instance, Snaeplayer works seamlessly without an internet connection. Furthermore, PWAs can access local files and support features like push notifications, provided the platform-specific apps allow it. They are truly cross-platform, adhering to open web standards, rather than being proprietary technology, making them a robust and inclusive solution for modern app experiences.

Since Progressive Web Apps operate directly within the browser, they inherently function within a sandboxed environment. This design prevents many direct exploits targeting OS APIs and the kernel, providing an additional layer of security compared to other technologies like app sandboxing.

2.5 WebAssembly

WebAssembly[16, 52] enables native languages like C and C++ to run on the web, achieving performance levels close to that of native applications. When combined with Progressive Web Apps, it simplifies the development of web-based applications using native languages, making the process more efficient and accessible.

2.6 Universal Windows Platform

Microsoft's earlier attempt to establish a walled garden ecosystem relied on its proprietary Universal Windows Platform (UWP) technology. However, Microsoft eventually abandoned UWP in favor of WinUI 3 and other frameworks. One of the key issues with UWP was its severe limitations, particularly due to the lack of a proper permission management system. For example, the developer of Rufus highlighted that UWP could not meet the requirements for accessing USB drives[49]. Even Microsoft's own documentation for Windows Terminal revealed challenges with UWP, indicating that it was unsuitable for implementation. Moreover, Microsoft's decision to restrict the use of C++ in UWP created additional hurdles, preventing apps like Spotify from porting their cross-platform core engines, which are built in C++ and Assembly, to the UWP ecosystem[11].

Although UWP was marketed as a cross-platform solution, developers seeking true cross-platform compatibility require a "universal platform" rather than a "universal Windows platform." They aim to develop for all major operating systems—such as Android, iOS, Linux, and macOS—beyond the Microsoft ecosystem. Progressive Web Apps have proven to be a superior alternative, offering all the capabilities UWP was meant to provide, while adhering to open web standards and supporting seamless functionality across platforms.

2.7 App Store and Sideloading

App Stores are now a standard feature across all major operating systems, including Linux GNOME, which offers Flatpak packages. As software distribution channels, App Stores, when properly managed, enhance security by having reviewers evaluate apps before their release—making them more secure than downloading random apps from the internet. However, the significant 30% revenue cut imposed by platforms like the Apple App Store and Google Play Store remains a widely criticized drawback. Additionally, some apps are either unavailable on these stores or are heavily modified to meet store requirements.

Microsoft is notably more accommodating toward Progressive Web Apps compared to Apple and Google, as they allow PWAs to be distributed through the Microsoft Store. Interestingly, when Steve Jobs unveiled the first-generation iPhone, there was no App Store; instead, he advocated for web apps as the primary application model. Within the framework of this paper, PWA apps should ideally be distributed via a public channel, such as a GitHub repository, while ensuring that a dedicated PWA App Store focuses exclusively on installing PWAs.

2.8 Antimalware Solutions, Including Microsoft Defender SmartScreen

Microsoft offers its own anti-malware solution integrated into the Microsoft Edge browser, such as Microsoft Defender SmartScreen, to guard against phishing attacks, malware, and downloading malicious files from the internet. However, much like C++ address sanitizers, these solutions cannot provide absolute guarantees. While they can help detect malicious actors, they cannot completely eliminate security threats. The persistence of malware issues on Windows demonstrates this limitation, despite Microsoft's deployment

of multiple layers of security measures, including SmartScreen, Windows Defender, and Windows Firewall.

Third-party anti-malware solutions also attempt to address these challenges, but they too have their own shortcomings. Notable incidents, such as the CrowdStrike outage, underscore the limitations of antivirus solutions across platforms, including Windows, Android, and macOS. These solutions often fail to completely prevent security threats and can sometimes introduce additional problems, as seen in cases like the aforementioned CrowdStrike outage. This highlights the inherent weaknesses in traditional antivirus approaches, especially in complex and evolving threat environments.

3 THREAT MODEL

We consider a scenario where users lack technical expertise and may unknowingly engage in risky online behaviors. A typical situation involves users browsing the web with Google Chrome, performing Google Searches, or mistyping URLs, which could inadvertently lead them to malicious or untrusted websites. Upon accessing these sites, users face multiple threats from malicious actors. These attackers may exploit vulnerabilities through techniques like side-channel attacks using JavaScript or WebAssembly, deliver cryptojacking, scareware[45], or leverage flaws in the browser's security.

Additionally, users can be tricked into downloading malicious applications designed to extract sensitive data, such as credit card information. Once this occurs, the attackers gain significant control, enabling them to execute harmful actions without restriction. Figure 1 illustrates the threat model, capturing the various attack vectors and their potential impacts.

4 DESIGN

The purpose of the S/P mode is to create a distinct separation between simple users and power users. This section details the design of the S/P mode toggle. Each Windows user account would offer the S/P mode toggle directly on the login screen. This toggle does not alter the existing permission management systems of Windows or Unix-based operating systems.

Figure 2 presents a prototype of the proposed login screen UI incorporating the S/P mode toggle. The key modification compared to the standard Windows login screen is the addition of a new button located on the right side of the existing login options. This button allows users to directly select either S Mode or P Mode from the login interface. The system remembers the user's previous choice: if S Mode was selected during the last session, it will default to S Mode; likewise, if P Mode was chosen, it will default to P Mode unless the user opts to switch. Notably, this enhancement is strictly a UI-level adjustment and does not alter the underlying functionality for user accounts, whether they are administrators, sudo users, or regular users. Instead, it introduces a secure and intuitive interface to complement the existing account structure without changing the way users interact with the system. When the user selects S Mode, the operating system does not launch the standard desktop UI. Instead, it uses Microsoft Edge as the engine for running PWAs and presents a UI similar to iPhone or Android, where users can access and launch PWAs in an intuitive, app-like interface. This design enhances accessibility while maintaining the simplicity of the PWA-focused environment.

4.1 S Mode

Figure 3 illustrates the architecture diagram for the S Mode toggle, while Figure 4 depicts the scope of the PWA Walled Garden. S Mode is specifically designed for users who focus on essential tasks such as working with Word, Excel, or engaging with social media apps like YouTube and Instagram, supported through progressive web apps. Advanced functionalities intended for power users are omitted in this environment. The primary goal of S Mode is to implement stringent controls, with restrictions surpassing those enforced on iPhones in numerous settings.

4.1.1 Disallow Direct Browser Use. Browsers are considered unsafe and are strictly prohibited for direct use. They are only permitted for hosting PWAs. No application, including Microsoft Edge or third-party browsers like Google Chrome and Firefox, should provide standalone browser functionalities.

4.1.2 Disallow Direct Search Engine Use. Search engines are also regarded as unsafe and are disallowed in S Mode. This restriction helps prevent phishing attempts, blocks access to fake websites, and mitigates the growing issue of SEO pollution[4]. Users are limited to searching only within the websites of installed PWAs, and are encouraged to rely on AI-powered chatbots, such as Microsoft Copilot, ChatGPT, or Deepseek, for accessing web content securely.

4.1.3 All Progressive Web Apps Must Be Installed via the App Store. Progressive web apps are installed in a manner similar to traditional apps on platforms like Google Play Store or Apple App Store. However, instead of native apps, users install PWAs. These apps run internally using Microsoft Edge, but direct access to the browser itself is restricted.

4.1.4 Browser Extensions are in the App Store. Since PWAs operate within browsers, they often leverage browser extensions. For example, users might install ad blockers for YouTube PWAs or use global speed/pitch adjusters to modify playback speeds for apps like Spotify, transforming them into nightcore players. To maintain control, such extensions should be made available only through the App Store.

4.1.5 Restrict Direct URL Access. Allowing users to type URLs poses risks, as typographical errors can lead to security issues. Hence, direct URL access is prohibited in all forms within S Mode.

4.1.6 PWA App Store Must Be Open. Progressive Web Apps are web resources and should remain open to ensure accessibility and transparency. Developers should have the ability to submit their PWAs to a publicly available GitHub repository, akin to how Microsoft vcpkg operates, or alternatively, rely on web standards to oversee PWA App Stores. Furthermore, no commission fees should be allowed for app submissions. This openness enables other operating systems, such as Linux and macOS, to adopt and implement this security model effectively.

4.1.7 Limit External Links. Web pages often include external links that could redirect users to malicious websites. To address this, PWAs must include a manifest file specifying a whitelist of allowed external links. By default, login pages for services like Google, Apple, or Microsoft accounts are permitted unless explicitly restricted by the app.

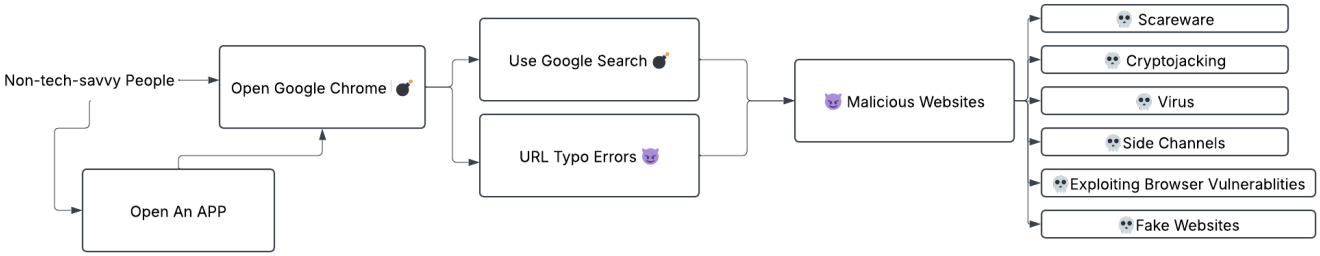

Figure 1: Threat Model: Browsers and Search Engines Considered Unsafe

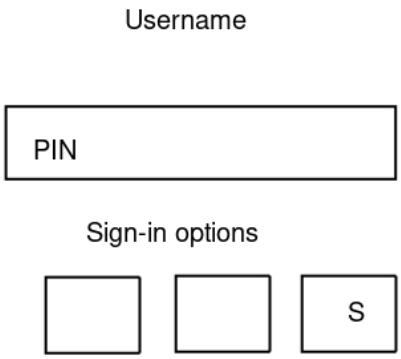

Figure 2: Demo of the S/P Mode Toggle Login Screen

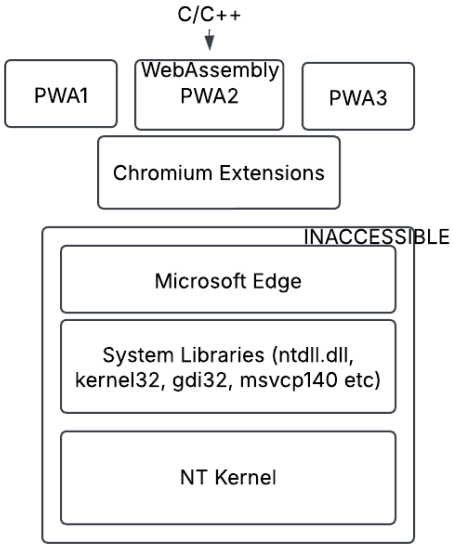

Figure 3: S Mode Toggle Architecture (NOT Windows S mode)

4.1.8 Restrict Permission Changes. Modern apps frequently request excessive permissions to exploit user data. In S Mode, users are prohibited from modifying app permissions to maintain data security.

4.1.9 Restrict Basic Settings Changes. Even seemingly simple settings, like date and time or timezone configurations, can be problematic for users. Incorrect settings often disrupt app functionality, such as the Google Play Store. To reduce complexity and maintenance needs, these settings cannot be altered in S Mode.

4.1.10 Restrict File Access to `$HOME/.smode` *Directory.* Complete restriction of file access is impractical, yet unrestricted access poses significant risks, as demonstrated by historical issues with Windows Phone. In S Mode, file access is limited to the `$HOME/.smode` directory (%USERPROFILE%/.smode on Windows), ensuring essential functionality while minimizing potential vulnerabilities. Additionally, users can synchronize their data seamlessly with Microsoft OneDrive within S Mode. However, apps should NEVER be allowed to access Microsoft OneDrive directly to prevent ransomware.

4.1.11 App Transitions Restricted to Installed PWAs. If one PWA attempts to open another PWA, the transition must be limited to installed PWAs. A browser popup should appear to handle the transition, but if the target PWA is not installed, the app transition should be blocked.

4.1.12 Gradually Implement Enhanced Security Measures for PWAs. Progressive Web Apps should gradually adopt stronger security measures to improve their robustness. For instance, WebAssembly binaries—often compiled from memory-unsafe languages like C/C++—require additional mitigations, such as WebAssembly Memory Tagging[51], to ensure safer execution. S Mode offers an opportunity to enforce stricter security measures for PWAs. However, as most WebAssembly applications currently lack adequate mitigations, this process must be introduced incrementally to allow for widespread adoption and compatibility.

4.1.13 S Mode UI Should Closely Resemble iPhone's UI. The iPhone's UI has become the de facto standard for mobile phone interfaces and walled garden operating systems. As mobile UIs are the most familiar to a majority of end users, adopting a similar design for S Mode ensures ease of use and eliminates any learning curve for users.

4.1.14 Automatic PWA Synchronization. Currently, PWAs are synchronized through Microsoft Edge, making them easily accessible for end users. To enhance usability in S Mode, these PWAs should be automatically synced, ensuring a seamless experience for users.

4.2 P Mode

P Mode represents the Windows environment in its standard form. It allows users to perform tasks such as running `.exe` files, utilizing browsers, accessing the Windows Subsystem for Linux, using `cmd`

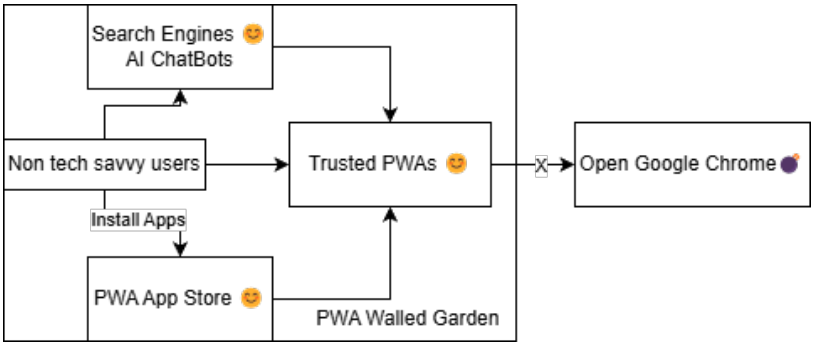

Figure 4: PWA Walled Garden in S Mode Toggle (NOT Windows S mode)

and PowerShell, editing the registry (`regedit`), and rebooting into UEFI to install other operating systems. Logging into P Mode signifies that the user is a power user, comparable to utilizing `unsafe` blocks in programming languages like C# or Rust. It conveys to the operating system: "I understand the risks and am fully capable of managing them. Please grant me unrestricted access to perform my tasks."

With the introduction of the S/P mode toggle, P Mode also takes on the responsibility of managing settings for S Mode. Below, we present the design principles for P Mode.

The Windows Settings app in P Mode should include a dedicated section for managing S Mode settings.

4.2.1 Power User Mode Must Grant Full Control, Including Installation of Other Operating Systems. P Mode is designed for power users and is not intended to be a locked-down environment. Power users should not face the restrictions common to ecosystems like Android or iOS. Instead, P Mode must allow users complete freedom, including the ability to install alternative operating systems.

4.2.2 Enable Comprehensive Settings Management. While S Mode restricts changes to date, time, and timezone settings, P Mode must provide full access to these configurations. This highlights the versatility of the S/P mode toggle compared to fully locked platforms like iPhones, where errors in settings can arise due to the absence of boundaries for unsafe operations. In the pursuit of complete safety, unintended vulnerabilities may emerge—this toggle offers a balance.

4.2.3 Allow installing of PWAs and Browser Extensions into S Mode. Since PWAs are essentially websites, installing them into S Mode should be straightforward—users can simply input URLs to initiate the process. Additionally, the Edge browser in P Mode can feature an option that allows users to install PWAs and Chromium extensions directly into S Mode.

4.2.4 Centralized Permission Management for PWAs in S Mode. Permissions for PWAs in S Mode should be managed exclusively through P Mode. Android apps often manipulate users into granting unnecessary permissions. By centralizing permission management in P Mode, this approach protects non-tech-savvy users from being tricked by applications while retaining full functionality for power users.

4.2.5 Adjust PWA Synchronization Settings. P Mode should offer users the flexibility to customize how PWAs are synchronized for S Mode. This ensures personalized and efficient use of PWAs.

4.2.6 Modify File Access Settings for S Mode. By default, file access in S Mode is restricted to `$HOME/.smode` (On Windows it is `%USERPROFILE%/.smode`) directory. In P Mode, users should be able to expand access by adding more directories or modifying the default directory that S Mode can access.

4.2.7 Dedicated File Storage for Each PWA. In addition to shared storage for Progressive Web Apps, each PWA can maintain its own dedicated file storage. This approach enhances sandboxing capabilities in S Mode, ensuring greater isolation and security for individual app.

4.2.8 Popup Suggestions for Entering S Mode in Sensitive Apps. Sensitive applications, such as banking PWAs like Chase, require heightened security measures. In this context, browsers operating in power user mode can provide a discreet suggestion prompting users to switch to S Mode for enhanced protection. To ensure a seamless user experience, these prompts should be limited in frequency and appear only for sensitive applications. Unless mandated by a bank's policy to enforce the use of S Mode, these suggestions should remain optional, allowing users to make the choice without feeling overwhelmed by intrusive notifications.

5 EVALUATION

Chromium's Rule of 2 states that no system should simultaneously combine untrusted input, unsafe implementation languages, and high privilege. We generalize this into a four-factor model suitable for operating systems and app ecosystems by adding a fourth dimension: patch latency. A system that is safe in theory but slow to update in practice remains vulnerable. We call this the **Rule of 4**:

(1) **Input trust** — whether the system accepts arbitrary, potentially malicious inputs.
(2) **Implementation safety** — whether applications are written in memory-safe languages.
(3) **Privilege level** — whether applications run with high or low system privilege.
(4) **Patch latency** — how quickly vulnerabilities can be fixed and deployed.

A secure system must avoid the dangerous intersection of all four. We evaluate S/P Mode using this Rule of 4 and validate the

model with four PWAs we implemented: the PWA Store[42], PWA Player[41], PWA Geolocation[40], and WasmPass[43]. All four applications are released as free and open source software under the GPLv3 license, ensuring full transparency, auditability, and user freedom. We then evaluate performance, usability, environmental impact, and compare S/P Mode to Android and iOS.

5.1 Security Evaluation Using the Rule of 4

5.1.1 Input Trust. S/P Mode removes the dominant source of untrusted input: arbitrary URLs, downloads, search results, and native binaries. All applications must be installed from the **PWA Store**, deployed on 2025-07-01. As of 2026-03-01, it contains over 1,000 PWAs. The store demonstrates:

- **Decentralization**: deployed on Cloudflare Pages with no servers.
- **Zero infrastructure cost**: it stores only URLs, not binaries.
- **Transparency**: each PWA lists its origin URL for public inspection.
- **Safety**: each PWA is scanned using ScamAdvisor.

S/P Mode differs fundamentally from Apple's App Store, Google Play, or Flatpak repositories, which host binaries, require heavy infrastructure, and obscure provenance. The PWA Store shows that a global app ecosystem can be maintained at near-zero cost while remaining transparent and auditable.

5.1.2 Implementation Safety. PWAs are written in HTML, CSS, and JavaScript—memory-safe languages. WebAssembly provides near-native performance while enforcing strict bounds checking and sandboxing.

To validate this, we implemented **WasmPass**, a WebAssembly port of the C++ fast_io password generator. Generating 10 million passwords showed no noticeable slowdown compared to native C++, while all memory accesses were checked by the WebAssembly runtime. Combined with recent work on WebAssembly memory tagging [**?**], this demonstrates that CPU-intensive workloads can be executed safely without exposing the system to C/C++ memory-safety vulnerabilities.

5.1.3 Privilege Level. PWAs run inside the browser sandbox, and in S/P Mode the browser itself is sandboxed by the OS. PWAs cannot access arbitrary files, kernel interfaces, or native APIs. They only receive capabilities explicitly exposed by the browser.

We validated this by implementing PWAs that use battery status, screen lock, geolocation, WebUSB, and WebBluetooth. These experiments show that PWAs can interact with real hardware while remaining fully sandboxed. For example:

- GrapheneOS can be flashed via WebUSB in Chromium browsers.
- An e-scooter can be controlled (e.g., beep) via WebBluetooth.
- Real-time device orientation, heading, speed, altitude, and high-accuracy GPS coordinates can be accessed through the Geolocation and DeviceOrientation APIs, as demonstrated by our GPLv3 PWA Geolocation application. It computes cardinal direction, multiple distance metrics, and reverse-geocoded place names entirely within the browser sandbox.

These capabilities require no native code and no elevated privilege.

5.1.4 Patch Latency. PWAs update automatically via service workers from their origin servers. The PWA Store is not the update channel; it only provides initial installation. This ensures:

- zero user intervention,
- no OS-level installers,
- no reboot requirements,
- immediate rollout of security fixes.

Browsers also update frequently and independently of the OS. In contrast:

- Android updates are delayed by OEMs and carriers,
- iOS updates are tied to device support lifecycles,
- both platforms lock bootloaders, preventing long-term maintenance.

S/P Mode does not restrict bootloaders or OS installation. Users can reinstall or replace the OS, run diagnostics (e.g., MemTest86), or install alternative systems such as Linux or FreeBSD. This extends device lifespan and reduces e-waste.

5.2 Security Case Study: PWA Player vs. MPV

To demonstrate the structural security benefits of PWAs, we compare our **PWA Player** with the native MPV media player. MPV is a high-quality open-source player, but like all native C/C++ applications, it has a history of memory-safety vulnerabilities, such as:

- **CVE-2021-30145**: format string vulnerability in M3U playlist handling, enabling code execution.
- **CVE-2020-19824**: arbitrary code execution via the `ao_c` parameter.

These vulnerabilities exist because MPV parses complex, attacker-controlled formats in an unsafe language at high privilege.

PWA Player, by design, cannot suffer from these classes of bugs:

- it is written in JavaScript (no buffer overflows or format-string primitives),
- it delegates parsing and decoding to the browser's hardened media pipeline,
- it runs inside the browser sandbox,
- it updates automatically via service workers.

The same crafted M3U file that could exploit MPV becomes harmless when passed to the browser's media engine. Entire categories of MPV CVEs are structurally impossible in PWA Player.

There is also a size contrast: MPV's Android APK (22.9 MB, SHA-256: b6926fa9...) bundles its own logic and decoders. PWA Player occupies only about 256 KB after installation on Android with Microsoft Edge, yet supports music, video, IPTV (M3U8), battery display, screenshots, and video recording—because these features are implemented by the browser engine, not by custom native code.

5.3 Performance Evaluation

Our three PWAs demonstrate that the PWA-based walled garden is not only safer but also performant:

- **PWA Store**: installation is instant; no APK downloads or OS-level installers.
- **PWA Player**: uses the browser's optimized media pipeline; we observed no perceptible playback degradation for IPTV or local media.

- **WasmPass**: shows that WebAssembly can handle tens of millions of operations with negligible slowdown relative to native C++.

Because PWAs share a single browser engine, they avoid the duplication and bloat of native frameworks and Electron-style apps. On Android, 54 installed PWAs plus browser cache consumed 3.06 GB total; the browser itself accounts for roughly 1 GB, leaving about 2 GB for all PWAs and their caches.

5.4 Usability Evaluation

S/P Mode is designed to provide a simple, predictable, and familiar experience while preserving full user autonomy. Its usability advantages arise from the S/P toggle and the separation between a locked-down environment (S/P Mode) and a fully open environment (P Mode). The key usability properties are:

(1) The S/P toggle appears directly on the login screen, giving users explicit control over whether they want a locked-down environment.
(2) S/P Mode runs only PWAs from the PWA Store. P Mode is the normal Windows environment capable of running any application.
(3) P Mode can install PWAs into S Mode, allowing power users to install PWAs directly from browsers.
(4) S/P Mode does not disable system services such as SSH servers or remote desktop.
(5) P Mode allows rebooting into UEFI, reinstalling other operating systems, and running diagnostic tools.
(6) S/P Mode can allow a limited set of signed native applications (e.g., Chrome, Firefox, WeChat, Android emulators, Microsoft Office) during a transition period.

S/P Mode is universally applicable to GNU/Linux, macOS, and FreeBSD. It does not depend on Windows-specific features; Windows is used only as a concrete example.

5.4.1 Comparison to iOS and Android. From a usability perspective, S/P Mode offers several advantages over iOS and Android:

- **Reversibility**: iOS and Android enforce a permanent walled garden. S/P Mode is optional and reversible.
- **Developer freedom**: iOS requires App Store approval; Android increasingly restricts sideloading. P Mode allows full developer workflows.
- **Device control**: iOS and Android prevent users from accessing bootloaders or reinstalling the OS. P Mode preserves full device control.
- **Application flexibility**: PWAs are cross-platform, lightweight, and automatically updated.
- **Transition path**: iOS and Android offer no migration path between locked-down and open environments. S/P Mode and P Mode coexist.

5.5 Environmental Impact

Because PWAs are small, memory-safe, and automatically updated, they extend device lifespan. S/P Mode does not lock bootloaders or prevent OS replacement, allowing users to maintain devices long after OEM support ends. In contrast, Android and iOS enforce planned obsolescence through locked bootloaders, proprietary firmware, and short update windows.

5.6 Rule-of-4 Comparison

Platform	Input	Lang.	Privilege	Patching
S/P Mode	10/10	10/10	10/10	10/10
iOS	3/10	6/10	2/10	3/10
Android	2/10	5/10	1/10	1/10

Table 1: Rule-of-4 comparison across platforms.

S/P Mode achieves perfect compliance with the Rule of 4 because it avoids untrusted input, uses memory-safe languages, runs at low privilege, and patches instantly. In contrast, iOS and Android score poorly due to permanent walled gardens, unsafe native code, elevated privileges for system components, and slow or fragmented patch delivery.

5.7 Summary

S/P Mode avoids the dangerous intersection of the Rule of 4. The PWA Store constrains inputs and demonstrates decentralized, zero-infrastructure distribution. PWA Player shows that entire classes of native CVEs are structurally impossible. WasmPass shows that CPU-intensive workloads can be executed safely and efficiently. S/P Mode is simpler, safer, more transparent, more repairable, and more environmentally sustainable than Android or iOS.

Finally, Windows is used only as a concrete example. The S/P Toggle is a general design pattern applicable to any non-walled-garden operating system. Once published, S/P Mode provides a technical and legal foundation for requiring platform vendors to support open, repairable, user-controlled computing environments.

6 RELATED WORK

Dynamic Malware Analysis in the Modern Era–A State of the Art Survey[32], authored by Or-Meir, Ori, Nissim, Nir, Elovici, Yuval, and Rokach, Lior, provides a definition of malware and categorizes it into various types, including viruses, trojans, spyware, worms, adware, scareware, bots, ransomware, and cryptominers. The paper also classifies malicious behaviors associated with malware. It offers a comprehensive survey evaluating the strengths and weaknesses of different analysis methods and their resilience against malware evasion techniques. In addition, the authors highlight how mainstream approaches predominantly rely on machine learning for malware detection. The paper *Analysis of Machine learning Techniques Used in Behavior-Based Malware Detection* by Firdausi, Ivan, Lim, Charles, Erwin, Alva, and Nugroho, Anto Satriyo employed the J48 decision tree algorithm, achieving a recall of 95.9%, a false positive rate of 2.4%, a precision of 97.3%, and an accuracy of 96.8%. *Malware Classification with Recurrent Networks*[33] by Pascanu, Razvan, Stokes, Jack W., Sanossian, Hermineh, Marinescu, Mady and Thomas, Anil explored malware detection using a recurrent bidirectional neural network to analyze system calls, with their best model yielding a precision of 82% and a false positive rate of 0.5% based on 114 distinct system calls from 250,000 malware samples. *A Machine-Learning Approach for Classifying and Categorizing Android Sources and Sinks*[38] by Rasthofer, Siegfried, Arzt, Steven, and

Bodden, Eric proposed the SUSI framework, evaluated on 11,000 malware samples, with their top-performing model achieving an accuracy of 92.3%. *Adaptive Detection of Covert Communication in HTTP Requests*[39] by Schwenk, Guido and Rieck, Konrad analyzed 695 malware samples but did not report metrics such as accuracy, precision, or false positive rate. *Trusted System-Calls Analysis Methodology Aimed at Detection of Compromised Virtual Machines Using Sequential Mining*[30] by Nissim, Nir, Lapidot, Yuval, Cohen, Aviad and Elovici, Yuval utilized memory dumps from virtual machines and reconstructed systems using WinDbg; their analysis of ten malware and six benign samples with random forest achieved a precision rate of 98% with no false positives. *Machine Learning in Side-Channel Analysis: A First Study*[21] by Hospodar, Gabriel, Gierlichs, Benedikt, De Mulder, Elke, Verbauwhede, Ingrid and Vandewalle, Joos demonstrated that machine learning could effectively extract cryptographic keys using power consumption traces as features, with their LS-SVM linear classifier reaching a success rate of 75%. *On the Feasibility of Online Malware Detection with Performance Counters*[10] by Demme, John, Maycock, Matthew, Schmitz, Jared, Tang, Adrian, Waksman, Adam, Sethumadhavan, Simha, and Stolfo, Salvatore tested their decision tree-based implementation on Android and Linux, using 201 benign samples and 503 malware samples, achieving a detection rate of 83% and a false positive rate of 10%. Finally, *EDDIE: EM-Based Detection of Deviations in Program Execution*[29] by Nazari, Alireza, Sehatbakhsh, Nader, Alam, Monjur, Zajic, Alenka and Prvulovic, Milos monitored electromagnetic emissions from embedded and IoT devices, detecting statistical anomalies caused by code injections into application loops.

Understanding and Detecting Real-World Safety Issues in Rust[37] by Qin, Boqin, Chen, Yilun, Liu, Haopeng, Zhang, Hua, Wen, Qiaoyan, Song, Linhai, and Zhang, Yiying highlights that programming languages like Rust can encounter memory safety issues if the `unsafe` keyword is misused. The authors conducted an extensive study, analyzing five widely-used Rust libraries and two online security databases. Their manual inspection uncovered 70 memory bugs, 100 concurrency bugs, and 110 programming errors leading to panics. This research emphasizes that excessive reliance on `unsafe` significantly diminishes its intended effectiveness.

Is Google Getting Worse? A Longitudinal Investigation of SEO Spam in Search Engines[4] by Bevendorff, Janek, Wiegmann, Matti, Potthast, Martin, and Stein, Benno examines the decline in Google Search quality due to the rise of SEO spam. The study highlights how spam and link farms are becoming increasingly indistinguishable, a trend that is expected to worsen with the advent of generative AI.

Bringing the Web Up to Speed with WebAssembly[17] by Barzolevskaia, Anna, Branca, Enrico, and Stakhanova, Natalia serves as a foundational work introducing WebAssembly by detailing its motivation, design, and formal semantics. *Not so Fast: Analyzing the Performance of WebAssembly vs. Native Code* by Jangda, Abhinav, Powers, Bobby, Berger, Emery D., and Guha, Arjun developed BROWSIX-WASM to evaluate the performance gap between WebAssembly and native code, finding an average slowdown of 45% on Firefox and 55% on Chromium, with peak slowdowns of 2.08x (Firefox) and 2.5x (Chrome). The performance issues were attributed to both platform-specific factors, such as bounds checking and function call verification, and missing optimizations. *Everything Old is New Again: Binary Security of WebAssembly*[25] highlights how WebAssembly binaries, often compiled from memory-unsafe languages like C/C++, remain vulnerable to memory safety exploits, enabling the construction of malicious cross-site scripting attacks. *WebAssembly Memory Tagging*[51] by Shengdun W. and Aravind P. offers a solution to WebAssembly's memory safety concerns using memory tagging, inspired by the ARM Memory Tagging Extension. Their research demonstrates that software-based memory tagging introduces overheads of 48.91% for Wasm64 and 72.38% for Wasm32, but with ARM MTE-supported CPUs, these overheads drop significantly to 5.71% for Wasm64 and 18.05% for Wasm32, while effectively addressing real-world CVEs.

7 CONCLUSION

S/P mode, akin to the `unsafe` keyword in programming languages like C# and Rust, is a straightforward yet highly effective concept. By leveraging progressive web applications and browser sandboxing, implementing an S/P mode toggle has the potential to mitigate most security risks associated with untrusted inputs. For non-technical users, a simple `walled garden` approach is essential to shield them from the inherent dangers of the internet.

8 FUTURE WORK

While S/P mode provides a protective `walled garden` for web browsers—currently the primary source of untrusted inputs—other vulnerabilities, such as malicious emails or SMS messages, may still bypass this safeguard. Extending the principles of S/P mode to address these challenges represents a promising direction for future research.

REFERENCES

[1] 2025. StatCounter Global Stats. (2025). https://gs.statcounter.com/ Accessed: 2025-03-23.

[2] Apple. 2021. Building a Trusted Ecosystem for Millions of Apps: A Threat Analysis of Sideloading. (2021). https://www.apple.com/privacy/docs/Building_a_Trusted_Ecosystem_for_Millions_of_Apps.pdf

[3] Apple. n.d.. App Sandbox - Apple Developer Documentation. (n.d.). https://developer.apple.com/documentation/security/app-sandbox Accessed: 2025-03-21.

[4] Janek Bevendorff, Matti Wiegmann, Martin Potthast, and Benno Stein. 2024. Is Google Getting Worse? A Longitudinal Investigation of SEO Spam in Search Engines. In *Advances in Information Retrieval: 46th European Conference on Information Retrieval, ECIR 2024, Glasgow, UK, March 24–28, 2024, Proceedings, Part III*. Springer-Verlag, Berlin, Heidelberg, 56–71. https://doi.org/10.1007/978-3-031-56063-7_4

[5] Jody Bruchon. n.d.. Windows 11 Must Be Stopped - A Veteran PC Repair Shop Owner's Dire Warning - Jody Bruchon. (n.d.). https://youtu.be/LcafzHL8iBQ Accessed: 2025-03-21.

[6] CBS Sunday Morning. 2024. The Tragic Costs of E-Waste. https://youtu.be/4GtWGHvX-rk. (2024). Accessed: 2026-03-03.

[7] European Commission. 2024. Commission Sends Preliminary Findings to Apple and Opens Additional Non-Compliance Investigation Against Apple Under the Digital Markets Act. (2024). https://ec.europa.eu/commission/presscorner/detail/en/ip_24_3433

[8] Reddit User Community. n.d.. What's wrong with snaps, why so many people hate it? (n.d.). https://www.reddit.com/r/linux/comments/j3ajnf/whats_wrong_with_snaps_why_so_many_people_hate_it/ Accessed: 2025-03-23.

[9] Wikipedia contributors. 2024. 2024 CrowdStrike-related IT outages. (2024). https://en.wikipedia.org/wiki/2024_CrowdStrike-related_IT_outages Accessed: 2025-03-21.

[10] John Demme, Matthew Maycock, Jared Schmitz, Adrian Tang, Adam Waksman, Simha Sethumadhavan, and Salvatore Stolfo. 2013. On the feasibility of online malware detection with performance counters. *SIGARCH Comput. Archit. News* 41, 3 (June 2013), 559–570. https://doi.org/10.1145/2508148.2485970

[11] EDGECELSIOR. 2024. Windows Phone WTF: EPISODE 7 - THE DREADED APP GAP, Microsoft banned C++ in UWP. (2024). https://youtu.be/-eF4Fdgi3KA Accessed: 2025-03-21.

[12] State Administration for Market Regulation. 2025. China's National Market Supervision Administration Decides to Investigate Google for Suspected Monopoly Law Violations. (2025). https://www.samr.gov.cn/xw/zj/art/2025/art_396a9ab3aa6d4c4bbd40833815afd245.html

[13] David Fortunato and Jorge Bernardino. 2018. Progressive web apps: An alternative to the native mobile Apps. In *2018 13th Iberian Conference on Information Systems and Technologies (CISTI)*. 1–6. https://doi.org/10.23919/CISTI.2018.8399228

[14] FreeBSDfan. 2024. We need a real GNU/Linux (not Android) smartphone ecosystem. (2024). https://www.reddit.com/r/linux/comments/1fx5fq0/we_need_a_real_gnulinux_not_android_smartphone/ Accessed: 27 March 2025.

[15] Samuel Greengard. 2025. Fighting for the Right to Repair. *Commun. ACM* (Sept. 2025). https://cacm.acm.org/news/fighting-for-the-right-to-repair Accessed: 2026-03-03.

[16] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the web up to speed with WebAssembly. *SIGPLAN Not.* 52, 6 (June 2017), 185–200. https://doi.org/10.1145/3140587.3062363

[17] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. 2017. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2017)*. Association for Computing Machinery, New York, NY, USA, 185–200. https://doi.org/10.1145/3062341.3062363

[18] Julie Haney. 2022. Users are not stupid: Six cyber security pitfalls overturned. (2022). https://csrc.nist.gov/csrc/media/Projects/usable-cybersecurity/documents/Final_Proof_Users_are_not_stupid.pdf Accessed: 2025-03-21.

[19] Julie Haney. 2023. Users Are Not Stupid: Six Cyber Security Pitfalls Overturned. (2023). https://csrc.nist.gov/csrc/media/Projects/usable-cybersecurity/documents/Final_Proof_Users_are_not_stupid.pdf

[20] Jerry Hildenbrand. 2021. Android will never be supported by the 'regular' Linux kernel, but that won't stop Google from trying. (2021). https://www.androidcentral.com/android-will-never-be-supported-regular-linux-kernel-thats-not-stopping-google-trying Accessed: 2025-03-21.

[21] Gabriel Hospodar, Benedikt Gierlichs, Elke De Mulder, Ingrid Verbauwhede, and Joos Vandewalle. 2011. Machine learning in side-channel analysis: a first study. *Journal of Cryptographic Engineering* 1, 4 (2011), 293–302.

[22] Jody Bruchon. 2021. Windows 11 Must Be Stopped - A Veteran PC Repair Shop Owner's Dire Warning. https://www.youtube.com/watch?v=LcafzHL8iBQ. (2021). Accessed: 2026-03-03.

[23] Keep Android Open Project. 2024. Keep Android Open. https://keepandroidopen.org. (2024). Accessed: 2026-03-03.

[24] Taeri Kim, Noseong Park, Jiwon Hong, and Sang-Wook Kim. 2022. Phishing URL Detection: A Network-based Approach Robust to Evasion. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS '22)*. Association for Computing Machinery, New York, NY, USA, 1769–1782. https://doi.org/10.1145/3548606.3560615

[25] Daniel Lehmann, Johannes Kinder, and Michael Pradel. 2020. Everything Old is New Again: Binary Security of WebAssembly. In *29th USENIX Security Symposium (USENIX Security 20)*. USENIX Association, 217–234. https://www.usenix.org/conference/usenixsecurity20/presentation/lehmann

[26] melontini. 2025. Bootloader Unlock Wall of Shame. (2025). https://github.com/melontini/bootloader-unlock-wall-of-shame

[27] Microsoft. 2024. Phishing trends and techniques - Microsoft Defender for Endpoint. (2024). https://learn.microsoft.com/en-us/defender-endpoint/malware/phishing Accessed: 2025-03-21.

[28] Microsoft. 2024. Prevent malware infection - Microsoft Defender for Endpoint. (2024). https://learn.microsoft.com/en-us/defender-endpoint/malware/prevent-malware-infection Accessed: 2025-03-21.

[29] Alireza Nazari, Nader Sehatbakhsh, Monjur Alam, Alenka Zajic, and Milos Prvulovic. 2017. EDDIE: EM-Based Detection of Deviations in Program Execution. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. Association for Computing Machinery, New York, NY, USA, 333–346. https://doi.org/10.1145/3079856.3080223

[30] Nir Nissim, Yuval Lapidot, Aviad Cohen, and Yuval Elovici. 2018. Trusted system-calls analysis methodology aimed at detection of compromised virtual machines using sequential mining. *Know.-Based Syst.* 153, C (Aug. 2018), 147–175. https://doi.org/10.1016/j.knosys.2018.04.033

[31] United States Department of Justice. 2024. Justice Department Sues Apple for Monopolizing Smartphone Markets. (2024). https://www.justice.gov/archives/opa/pr/justice-department-sues-apple-monopolizing-smartphone-markets

[32] Ori Or-Meir, Nir Nissim, Yuval Elovici, and Lior Rokach. 2019. Dynamic Malware Analysis in the Modern Era—A State of the Art Survey. *ACM Comput. Surv.* 52, 5, Article 88 (Sept. 2019), 48 pages. https://doi.org/10.1145/3329786

[33] Razvan Pascanu, Jack W. Stokes, Hermineh Sanossian, Mady Marinescu, and Anil Thomas. 2015. Malware classification with recurrent networks. In *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 1916–1920. https://doi.org/10.1109/ICASSP.2015.7178304

[34] Android Vulnerabilities Project. n.d.. Android Vulnerabilities by Year. (n.d.). https://androidvulnerabilities.org/by/year/ Accessed: 2025-03-21.

[35] Chromium Project. n.d.. The Rule of 2 - Chromium Security Documentation. (n.d.). https://chromium.googlesource.com/chromium/src/+/master/docs/security/rule-of-2.md Accessed: 2025-03-23.

[36] GNU Project. 2025. Tivoization. (2025). https://www.gnu.org/philosophy/tivoization.html Accessed: 27 March 2025.

[37] Boqin Qin, Yilun Chen, Zeming Yu, Linhai Song, and Yiying Zhang. 2020. Understanding memory and thread safety practices and issues in real-world Rust programs. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 763–779. https://doi.org/10.1145/3385412.3386036

[38] Siegfried Rasthofer, Steven Arzt, and Eric Bodden. 2014. A machine-learning approach for classifying and categorizing android sources and sinks. In *NDSS*, Vol. 14.

[39] Guido Schwenk and Konrad Rieck. 2011. Adaptive Detection of Covert Communication in HTTP Requests. In *2011 Seventh European Conference on Computer Network Defense*. 25–32. https://doi.org/10.1109/EC2ND.2011.12

[40] S/P Mode Project. 2026. PWA Geolocation. https://pwa-geolocation.pages.dev. (2026). Accessed: 2026-03-03.

[41] S/P Mode Project. 2026. PWA Player. https://pwa-player.pages.dev. (2026). Accessed: 2026-03-03.

[42] S/P Mode Project. 2026. PWA Store. https://pwa-store-7x5.pages.dev. (2026). Accessed: 2026-03-03.

[43] S/P Mode Project. 2026. WasmPass. https://wasmpass.pages.dev. (2026). Accessed: 2026-03-03.

[44] Microsoft Support. Access Year. Windows 10 and Windows 11 in S mode FAQ. (Access Year). https://support.microsoft.com/en-us/windows/windows-10-and-windows-11-in-s-mode-faq-851057d6-1ee9-b9e5-c30b-93baebeebc85 Accessed: 27 March 2025.

[45] Microsoft Edge Team. 2025. Stand up to scareware with scareware blocker. (2025). https://blogs.windows.com/msedgedev/2025/01/27/stand-up-to-scareware-with-scareware-blocker/ Accessed: 2025-03-21.

[46] Chris Titus Tech. 2022. Ubuntu's Decline. (2022). https://www.youtube.com/watch?v=pMfqCzbSmQU Accessed: 2025-03-23.

[47] Peter "Durante" Thoman. 2016. Why PC games should never become universal 'apps'. (2016). https://www.pcgamer.com/why-pc-games-should-never-become-universal-apps/ Accessed: 2025-03-21.

[48] U.S. Federal Trade Commission. 2019. Nixing the Fix: Workshop on Repair Restrictions. https://www.ftc.gov/news-events/events/2019/07/nixing-fix-workshop-repair-restrictions. (2019). Accessed: 2026-03-03.

[49] GitHub user. 2020. Why do you hate UWP? Seems promising thing. (2020). https://github.com/pbatard/rufus/issues/1617 Accessed: 2025-03-21.

[50] Reddit user. n.d.. Creator of Rufus outlines the problems with Microsoft's UWP. (n.d.). https://www.reddit.com/r/programming/comments/noagrx/creator_of_rufus_outlines_the_problems_with/ Accessed: 2025-03-21.

[51] Shengdun W. 2024. Public Presentation for WebAssembly Memory Tagging. (25 November 2024). https://www.youtube.com/live/8W9faYGooqA Streamed live, 64 views as of access. No description provided. Accessed: 2025-03-23.

[52] WebAssembly. 2025. WebAssembly. https://webassembly.org/. (2025).

[53] web.dev. n.d.. Progressive Web Apps - Learn PWA. (n.d.). https://web.dev/learn/pwa/progressive-web-apps?hl=fr Accessed: 2025-03-21.

[54] Tim Werthmann, Ralf Hund, Lucas Davi, Ahmad-Reza Sadeghi, and Thorsten Holz. 2013. PSiOS: bring your own privacy & security to iOS devices. In *Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security (ASIA CCS '13)*. Association for Computing Machinery, New York, NY, USA, 13–24. https://doi.org/10.1145/2484313.2484316

[55] World Economic Forum. 2026. 8 predictions for the world in 2030. https://x.com/wef/status/983378870819794945. (2026). Accessed: 2026-03-03.